CHAPTER 5

NOSQL DATABASE

INTRODUCTION

Since the early days of web development, programmers have used databases to store data safely and keep everything consistent (a “single source of truth”).

At first, most systems used relational databases (RDBMS) because they are strong and reliable (online trading), where transactions must be accurate.

Databases can work with different types of system designs (2-tier, 3-tier, etc.), and over time they were also used in newer applications like Web 2.0 and cloud-based apps.

When choosing a database, developers usually debated between open-source options (like MySQL) and paid/vendor options (like Oracle).

The discussion focused mostly on cost and trust, not on whether the database should be relational or a different type

INTRODUCTION

Earlier, most computing courses taught only relational databases (RDBMS), so developers naturally used them for all database problems.

When large web platforms like Facebook and Twitter grew, they needed to handle huge amounts of data across the world.

Relational databases struggled with this because: They don’t scale easily across many global servers. They rely on ACID rules, which ensure data safety but slow things down

These new systems often didn’t need strict transactions, so the extra overhead caused poor performance. Retrieving data quickly from many locations also became difficult. Companies like Google solved this by building their own systems, like BigTable, which is not relational but designed for massive distributed data. Similar open-source systems (like HBase) were later developed.

Relational Database

- Relational databases (RDBMS) have been compelling and important.
- Many companies have invested huge amounts of money in them, so they won't replace them quickly.
- In the 1980s, getting data was slow and complicated. But with personal computers and SQL, people could access data quickly and make better decisions.
- Technology has changed a lot over time (like the rise of the Windows operating system and modern apps), so databases also need to evolve.
- Today, the three most important factors for databases are the following:
    - Scalability (handle more data or users)
    - Availability (system uptime)
    - Performance (speed)
- Availability used to be a big focus, but now most systems are already very reliable, so it's less of a difference.

NoSQL Database

- In scalability and performance, where NoSQL databases claim to be better than traditional relational databases.
- Modern systems often deal with huge, unstructured data spread across many locations (like web data).
    - Relational databases maintain strict ACID rules and use complex operations like joins
    - Relational databases usually do vertical scaling (one powerful machine with more CPU, RAM, etc.), whereas NoSQL databases use horizontal scaling (many smaller machines working together)
- In NoSQL, data is distributed across machines using a method called sharding (splitting data based on keys).

Relational databases are still useful and widely used, but modern big-data systems often prefer NoSQL because it is faster and scales better across many machines.

TYPES OF NOSQL

- NoSQL means any database that doesn’t use SQL, but this isn’t fully correct because other query languages (like OQL) already existed.
- SQL is very popular, so some NoSQL databases are actually adding SQL-like features to make them easier to use. For example, Apache Cassandra introduced CQL (Cassandra Query Language).
- Thus, NoSQL is defined as the following:
  - A database that does not follow the relational model, and
  - Uses query languages different from standard SQL (ISO standard)
- Common types of NoSQL include:
  - Column-based databases
  - Document-based databases

Databases that are non-relational and use different ways (not standard SQL) to store and query data.

CAP APPROACH

- Modern systems deal with huge amounts of data spread across many servers, and this is where traditional relational databases (RDBMS) start to struggle.
- In RDBMS: Indexes help find data faster, but they add extra work and slow things down when data changes often. Data is stored in multiple tables, so joins are needed, which are also slow and complex. ACID rules ensure data is correct and consistent, but enforcing them across many servers takes time and can lock data, reducing performance.
- Brewer's CAP Theorem states that a distributed database system can only guarantee two of the three following properties simultaneously:
- Consistency: All clients across all nodes see the same data at the same time.
- Availability: The system always responds to requests (even if some data might be stale).
- Partition Tolerance: The system continues operating even if some network connections between nodes break.

CAP

- The chapter makes an important clarification: the consistency in CAP is different from the consistency in ACID.
- ACID consistency means data complies with all business rules and constraints. 'CAP consistency' means all nodes hold identical data at the same moment.
- Traditional RDBMS typically prioritise Consistency and Availability (CA) and sacrifice partition tolerance.
- NoSQL systems typically prioritise Availability and Partition Tolerance (AP) and trade away strict consistency.

CAP

- Availability → The database is always ready to give data to users. More copies of data (on different servers) = higher availability. But too many copies can reduce performance
- Partition Tolerance → The system keeps working even if there are network failures (servers can’t communicate properly)
- In many NoSQL systems, perfect consistency is not always required:
  - Data might not be identical everywhere at all times
  - Some validation is handled by the application (client), not the database

Relational databases become slow with massive distributed data because of indexes, joins, and strict ACID rules.

The CAP theorem shows that modern systems must balance consistency, availability, and fault tolerance, which is why NoSQL databases often relax strict consistency for better performance.

TRADEOFFS – DISCUSSION BOARD SCENARIO

- There are two servers (Network 1 & 2) storing the same data. A user posts a question, and others reply. Due to a network failure. Some replies are copied to both servers. One reply (M) is only on one server
- Result: Some users don’t see reply M
- The system is:
  - Still working (available)
  - Still handling network failure (partition-tolerant)
  - But not fully consistent (data differs between servers)
- If you want perfect consistency (same data everywhere). You must wait until all servers sync. This slows down the system. If you want fast and always available systems: You allow temporary differences in data (stale data)

TRADE OFFS

- Banking / shopping systems → need strict accuracy (ACID, consistency)
- Discussion boards / social apps → can accept small delays or missing updates
- Also, strict systems may lock data while updating, making users wait. Some users prefer slightly outdated data instead of waiting

In distributed systems, you must choose a balance: perfect accuracy, always-available service, and handling network failures. You can’t have all three perfectly at the same time, so the choice depends on the application.

BASE

- Where RDBMSs are governed by ACID, most NoSQL databases operate under BASE. Main idea behind most NoSQL databases, similar to how ACID is used in relational databases.
- In ACID systems, data is always consistent and reliable immediately.
- In BASE systems: Data is not always instantly consistent but it will become consistent eventually, called 'eventual consistency'.
- BASE defines:
    - Basically Available: The system guarantees availability, though responses may be stale or inconsistent.
    - Soft State: The state of the system may change over time even without new input
    - Eventually consistent: All copies of the data will, at some point, be the same just not necessarily right now.

How NoSQL handles consistency

BASE consistency can be managed in two key ways:

Read Repair: When a read operation detects old data on a node (via timestamp comparison), it refreshes that data immediately. (When data is read, the system checks for outdated copies; if found, it updates them automatically.)

Delayed Repair (Weak Consistency): The system returns whatever value it finds, even if old, but marks it for a later update. (The system may return old data. It marks it for updating later. This improves performance and speed.

ACID (RDBMS) gives strict accuracy and slower performance, while BASE (NoSQL) gives faster performance, but less immediate accuracy

Many NoSQL systems let you adjust how much consistency you want depending on the application. A trade-off allows NoSQL databases to be faster and more scalable, making them suitable for modern large-scale applications.

Traditional(Row- Based) Databases

- Traditional RDBMS stores data in rows where all fields of a record stored together.
- Most databases store data in rows (like tables or spreadsheets)
- This works well when you need complete records (e.g., all details about one user).
- Techniques like normalization and indexing help improve performance.
- But for analytical workloads where you might want to compute the average or sum of one particular column across millions of rows, the RDBMS has to read every row and they ignore unnecessary data. It makes it slow for large datasets.
- These type of database are best for Transaction systems (banking, shopping), as frequent inserts and updates.

Column-Based Database

- Store data column by column instead of row by row. Thus all values of one column are stored together
- Faster reading for column-based queries and better compression (similar data stored together)
- Great for analytics and decision-making systems
- Slow for writing inserting data Data may need to be rearranged. Slow for reconstructing full rows
- Performance can drop if data changes frequently (less efficient compression)

Column-Based Database

- Store data column by column instead of row by row. Thus, all values of one column are stored together. In the column-based model, all values for one attribute are stored consecutively on disk. This provides:
  - Fast column scans: No jumping around — just a simple start-to-end read pointer.
  - Better compression: Similar data values sitting adjacent compress far better.
- The trade-off is that inserting or updating records becomes expensive, since data may need to be shuffled to keep similar values adjacent.
- Column-based databases are therefore best suited for:
  - Decision Support / Analytics: High-speed reads on specific columns, relatively static data.
  - Business Intelligence: Marketing analytics, trend analysis, and aggregates.
- They are a poor fit for high-volume transactional systems with many inserts and updates.
- These type of database are good for Data that doesn’t change often
- Example: Apache Cassandra is a column-based database.

Apache Cassandra (Column-Based)

Cassandra works very differently from traditional relational databases. Each piece of data is stored as a key paired with a value. Unlike simple key-value stores, Cassandra allows these key-value pairs to be nested, so we can store more complex data.

Cassandra's Basic Building Block

Keyspace: Like a database schema in RDBMS. Groups related column families. Has a placement strategy to decide how data replicas are distributed across nodes for horizontal scaling (add more machines, preferred in NoSQL)

- Column Family: Similar to a table in RDBMS. Stored in a separate file, sorted by key. Columns that are often accessed together should be stored together for better performance.
- Column: The smallest unit of storage.
    - Name (key) – identifies the row in the column family; Value – the actual data

Apache Cassandra (Column-Based)

- Timestamp – used for conflict resolution in distributed systems (usually in milliseconds or microseconds).
- SuperColumn: A container for multiple columns with related data. Similar in concept to a view in RDBMS.
- A SuperColumn called 'TransactionID' which contains those attributes, each stored as columns.
- A SuperColumn does not have a timestamp.
- Timestamps are on the individual columns inside it.

NOTE: Keyspace → Column Family → (SuperColumn → Columns) forms the hierarchical structure of Cassandra's data model.

Cassandra Query Language (CQL)

A higher-level Cassandra Query Language (CQL) was introduced which was modelled on standard SQL to lower the barrier for database professionals already familiar with SQL.

Notably, CQL allows the use of CREATE TABLE as an alias for CREATE COLUMN FAMILY, reflecting the overlap in terminology.

A CQL script: Creating a keyspace with a defined replication strategy and factor

Creating a column family using SQL-like CREATE TABLE syntax

Bulk-importing CSV data using the COPY command

Using SELECT * and COUNT(*) with WHERE clauses

Cassandra Query Language (CQL)

The tutorial also covers timing queries using the Linux TIME command, setting up performance benchmarking comparisons with MySQL.

The replication strategy in CQL is important: SimpleStrategy with replication_factor = 1 is used for standalone testing, but in production, higher replication factors across multiple nodes are used to improve availability and fault tolerance.

Document-Based Approach

The document-based model represents a fundamentally different philosophy to both relational and column-based storage. Its core characteristics are:

- **Schemaless**: There is no fixed schema. Two documents in the same collection can have completely different fields.
- **Flexible structure**: Fields can be arrays, sub-documents (nested objects), or simple values.
- **No schema validation at the database layer**: Data quality is the application developer's responsibility.

The flexibility is deliberate. As the chapter notes, much real-world data is *semi-structured – web* page content, event logs, user-generated content – and forcing it into a rigid relational schema requires constant schema changes and complex JOIN operations.

Document Based Approach

Document databases sacrifice transactional locking for performance. They are best suited for:

- Large volumes of semi-structured data
- Web application data (comments, user profiles, event logs)
- Rapid development where schema evolves frequently

Sharding in the document-based model is "Shared Nothing": each node runs its own database instance independently. When data is needed, the system looks up the relevant shard. Replication of shards is built in to handle node failures

Document-Based Approach

- Schemaless Design – Unlike RDBMS, there is no fixed schema. The application developer is responsible for data integrity and quality, rather than relying on centralised constraints. This enables flexibility: adding new fields or changing structures is simple.
- Performance Benefits: RDBMS spends time enforcing constraints and ACID rules. Document databases reduce overhead by avoiding strict transactional locks. This allows faster writes, especially for large volumes of semi-structured data such as web content, logs, or comments.
- Use Cases: Best suited for semi-structured or unstructured data. Examples: web content, social media comments, event logs, analytics datasets. Not ideal for transaction-heavy applications that require strict ACID compliance.
- Sharding and Horizontal Partitioning: Large datasets can be distributed across multiple nodes (shared-nothing architecture). Each node has its database instance, which allows parallel processing. Data is often replicated to maintain availability and protect against node failure.

Document Based Approach

- Documents are stored in collections, not tables. Each document can have different fields and nested structures (arrays and sub-documents). Example documents:

```
{
name:{ First: “Penelope”, Surname: “Pitstop” },
Birthday: new Date(“Jun 23, 1912”),
RacesWon: [ “Alaska”, “Charlo esville”]
}
{
name: { First: “Peter”, Surname: “Pefect” },
Birthday: new Date(“May 23, 1940”),
Hometown: “Miami”
Favourite: “Penelope”
}
```

- These two pieces of information do not look similar enough to become rows in a standard RDBMS. They don’t share all fields for a start. And yet a document-based approach allows that sort of flexibility. The name field in both cases is a container for other fields. This is called document embedding. Embedded documents can be complex. RacesWon, however, with its square brackets, is an array. Document databases prioritize flexibility and performance over strict consistency and normalization, making them ideal for modern distributed, large-scale applications

MongoDB (Document-Based Approach)

- MongoDB chooses to store its data following the JavaScript Object Notation (JSON) rules. JSON is a data-interchange format. For speed this human-readable format is turned into Binary format and stored as BSON.
- Documents are stored in BSON (Binary JSON) for efficiency, even though the developer usually works with JSON-like syntax. Embedded documents allow hierarchical or nested data without needing separate tables or explicit joins. Collections are schema-less, so documents in the same collection can have different fields.
- A MongoDB database is a collection of related data, just as in an RDBMS
- A MongoDB collection is a container for documents. It can be thought of as RDBMS table-like
- A MongoDB document is rather like a RDBMS row.
- A MongoDB field is rather like an RDBMS column.
- A Mongo embedded document is rather like a RDBMS join.
- A MongoDB primary key is the same as a RDBMS primary key.
- A MongoDB secondary index is the same as a RDBMS secondary index.

MongoDB Vs RDBMS

MongoDB	RDBMS	Notes
Database	Database	A logical container for related data.
Collection	Table	Holds documents, like a table holds rows.
Document	Row	Each document is an individual record.
Field	Column	Represents an attribute of a record.
Embedded Document	Join	Nested data inside a document, analogous to related rows in another table.
Primary Key	Primary Key	_id in MongoDB uniquely identifies a document.
Secondary Index	Secondary Index	Speeds up queries on non-primary key fields.

Thankyou!